



UNIVERSIDADE KIMPA VITA
INSTITUTO POLITÉCNICO
CURSO DE ENGENHARIA INFORMÁTICA

RELATÓRIO TÉCNICO DE LÓGICA DE PROGRAMAÇÃO II

Sistema de Gestão de Estudantes

Por:

Adilson Eduardo Ernesto Francisco

Afonso Pedro

António Armando Miranda

Marcos David António

Silva António Zangui

Grupo nº: 13

Iº Ano

Turma: TL 101

Período: Diurno

Orientado Pelo Docente:

Moyo Kanivengidio, MSc.

Uíge, Maio de 2026

1. INTRODUÇÃO

A gestão eficiente de informações é uma necessidade fundamental em qualquer sistema informático moderno. Nas instituições de ensino, o armazenamento, organização e consulta de dados dos estudantes são tarefas frequentes que exigem mecanismos eficientes de processamento.

Neste contexto, foi desenvolvido um Sistema de Gestão de Dados de Estudantes em linguagem C, utilizando estruturas de dados, algoritmos de ordenação, algoritmos de pesquisa e técnicas de manipulação de ficheiros.

O sistema permite carregar informações de estudantes a partir de um ficheiro de texto, organizar os dados segundo diferentes critérios, efetuar pesquisas rápidas e armazenar os resultados em ficheiros de saída.

2. OBJETIVOS

Objetivo Geral

- ✓ Desenvolver um sistema para gestão de dados de estudantes aplicando conceitos fundamentais de Estruturas de Dados e Algoritmos em linguagem C.

Objetivos Específicos

- ✓ Estudar os principais conceitos da linguagem C;
- ✓ Compreender e implementar as estruturas de repetição, estruturas homogêneas de dados e funções;
- ✓ Entender e implementar os algoritmos de ordenação e de pesquisa;
- ✓ Compreender e implementar a manipulação de arquivos em C;
- ✓ Conceber e implantar o sistema.

3. ANÁLISE DOS REQUISITOS

3.1 Requisitos Funcionais

O sistema permite:

RF1 – Leitura de Dados: Ler informações dos estudantes a partir do ficheiro: estudantes.txt

RF2 – Visualização: Mostrar todos os estudantes carregados em memória.

RF3 – Ordenação: Ordenar os estudantes por: Matrícula, Nome e Nota.

Utilizando os algoritmos de ordenação: Bubble Sort, Selection Sort e Insertion Sort.

RF4 – Pesquisa: Pesquisar estudantes por: Matrícula e Nome.

Utilizando: Pesquisa Linear e Pesquisa Binária

RF5 – Gravação: Guardar resultados em: Ficheiros de texto e Ficheiros binários

RF6 – Interface: Disponibilizar um menu interativo para o utilizador.

3.2 Requisitos Não Funcionais

Usabilidade: O sistema deve possuir uma interface simples e intuitiva.

Desempenho: Os algoritmos executam corretamente mesmo para conjuntos maiores de dados.

Manutenibilidade: O código deve estar organizado em funções para facilitar futuras alterações.

Confiabilidade: O sistema faz verificação de erros de leitura e escrita de ficheiros.

4. ESTRUTURA DA SOLUÇÃO DESENVOLVIDA

Os dados dos estudantes são representados através da seguinte estrutura:

```
typedef struct
{
    int matricula;
    char nome[100];
    float nota;
} Estudante;
```

Cada estudante possui: número de matrícula, nome completo e nota final.

Os estudantes são armazenados num vetor: *Estudante estudantes[100]*;

O programa foi dividido em funções independentes para aumentar a organização e reutilização do código.

Principais funções implementadas:

```
lerFicheiro();
mostrarEstudantes();
bubbleSort();
selectionSort();
insertionSort();
pesquisaLinear();
pesquisaBinaria();
gravarTexto();
gravarBinario().
```

5. ALGORITMOS DE ORDENAÇÃO IMPLEMENTADOS

5.1 Bubble Sort

O Bubble Sort compara elementos adjacentes e troca-os quando estão fora de ordem.

Complexidade: **$O(n^2)$**

Justificativa: escolhido pela facilidade de compreensão e simplicidade de implementação.

5.2 Selection Sort

O Selection Sort procura o menor elemento da parte não ordenada e posiciona-o corretamente.

Complexidade: **$O(n^2)$**

Justificativa: foi escolhido por realizar menos trocas do que o Bubble Sort.

5.3 Insertion Sort

O Insertion Sort insere cada elemento na posição correta dentro da parte já ordenada.

Complexidade:

Pior caso: $O(n^2)$

Melhor caso: $O(n)$

Justificativa: apresenta bom desempenho quando os dados já estão parcialmente ordenados.

6. ALGORITMOS DE PESQUISA IMPLEMENTADOS

6.1 Pesquisa Linear

Percorre os elementos sequencialmente até encontrar o valor procurado.

Complexidade: $O(n)$

Justificativa: funciona mesmo sem ordenação prévia.

6.2 Pesquisa Binária

A Pesquisa Binária divide sucessivamente o conjunto de dados ao meio. Requer que dados estejam ordenados para funcionar.

Complexidade: $O(\log n)$.

Justificativa: oferece melhor desempenho em listas ordenadas.

7. MANIPULAÇÃO DE FICHEIROS

O sistema utiliza ficheiros para armazenamento permanente dos dados.

Leitura:

fopen();

fscanf();

fclose().

O ficheiro de entrada possui o formato:

001; Adilson Francisco;18.5

002;Marcos David;15.0

003;Silva Zangui;17.0

Escrita em Texto: *fprintf()*.

Escrita Binária: *fwrite()*.

Tratamento de Erros

Antes de qualquer operação é feita a verificação:

if(fp == NULL)

{

printf("[x] Erro ao abrir ficheiro");

}

Garantindo maior robustez do sistema.

8. MANUAL DE UTILIZAÇÃO

Compilação

O código pode ser compilado utilizando um dos seguintes compiladores: **Dev-C++**, **Code::Blocks**, **VS Code**, ou outro compilador de códigos escritos em linguagem C.

Execução

Para a sua eficiente execução, o arquivo “SIGESTU.c” deve estar na mesma pasta que o arquivo “estudantes.txt”.

Menu Principal

[1] > Ler ficheiro

[2] > Mostrar estudantes

[3] > Ordenar

[4] > Pesquisar

[5] > Gravar texto

[6] > Gravar binario

[0] > Sair

Utilização:

Selecionar:

- 1: permite carregar os estudantes;
- 2: permite visualizar os dados;
- 3: permite escolher um método de ordenação;
- 4: permite realizar pesquisas;
- 5: permite guardar os resultados em arquivo de texto;
- 6: permite guardar os resultados em arquivo binário;
- 7: encerra a execução do programa.

9. TESTES E RESULTADOS

Foram realizados os seguintes testes:

TESTE	RESULTADO
Leitura do ficheiro	Sucesso
Visualização dos dados	Sucesso
Ordenação (Bubble, Selection e Insertion)	Sucesso
Pesquisas(Linear e Binária)	Sucesso
Escrita(Texto e Binário)	Sucesso
Tratamento de erros de Ficheiros	Sucesso

Fonte: Autores.

Os resultados demonstraram o correto funcionamento de todas as funcionalidades previstas.

CONCLUSÃO

O desenvolvimento deste projeto permitiu consolidar conhecimentos importantes sobre Estruturas de Dados e Algoritmos na linguagem C.

Durante a implementação foram aplicados conceitos de:

Estruturas (struct);

Vetores;

Strings;

Funções;

Manipulação de ficheiros;

Algoritmos de ordenação;

Algoritmos de pesquisa;

Programação modular.

Os principais desafios encontrados estiveram relacionados com a leitura correta de ficheiros, implementação dos algoritmos de ordenação e manipulação de strings.

Conclui-se que os objetivos propostos foram alcançados com sucesso, resultando num sistema funcional capaz de gerir dados de estudantes de forma eficiente.